Institute Of Engineering And Management

TERM-II PROJECT REPORT

Hospital Appointment System with Priority-Based Scheduling and Smart Slot Suggestion

Course Name	Advanced Object-Oriented Programming
Project Title	Hospital Appointment System with Priority-Based Scheduling and Smart Slot Suggestion
Student Name	Abhirup Pal
Roll Number	**03**
Enrollment Number	**12024052004002**
Semester	4th Semester
Department	Information Technology

This report documents the analysis, design, implementation, testing, and development workflow of a console-based Java application developed using a modular object-oriented structure. The system was designed to remain beginner-friendly while still demonstrating a clearly differentiated feature set through appointment priority handling and automatic slot suggestion.

1. Problem Statement

Hospitals frequently face appointment-management problems such as manual scheduling mistakes, repeated paperwork, unclear prioritization of urgent cases, and inefficient handling of occupied time slots. When booking is done informally, the same doctor and slot may be assigned to more than one patient, or genuinely urgent patients may be delayed because the system does not distinguish their importance from routine cases.

The aim of this project is to build a small but meaningful hospital appointment system that automates registration, appointment booking, cancellation, and viewing of hospital records. The system should be simple enough to be implemented in core Java, yet structured enough to reflect clean object-oriented design and modular code organization.

A distinguishing requirement of this project is the inclusion of a special feature that many basic student projects do not provide. For this reason, the system introduces **priority-based scheduling** and **smart next-slot suggestion**. Emergency patients receive the highest priority, senior citizens receive the next level of preference, and normal patients are handled as routine cases. If a preferred slot is already occupied, the system either suggests the next available slot or reschedules a lower-priority booking when appropriate.

Therefore, the project addresses both an operational problem and a software-design problem. Operationally, it reduces conflict during booking. From a software perspective, it demonstrates how classes, objects, encapsulation, modularity, file persistence, and service-oriented logic can be combined into a working Java application.

2. System Design

2.1 UML Class Diagram

Figure 1 shows the corrected UML class diagram for the implemented design. The diagram focuses on actual classes present in the source folder and the relationships that exist between them.

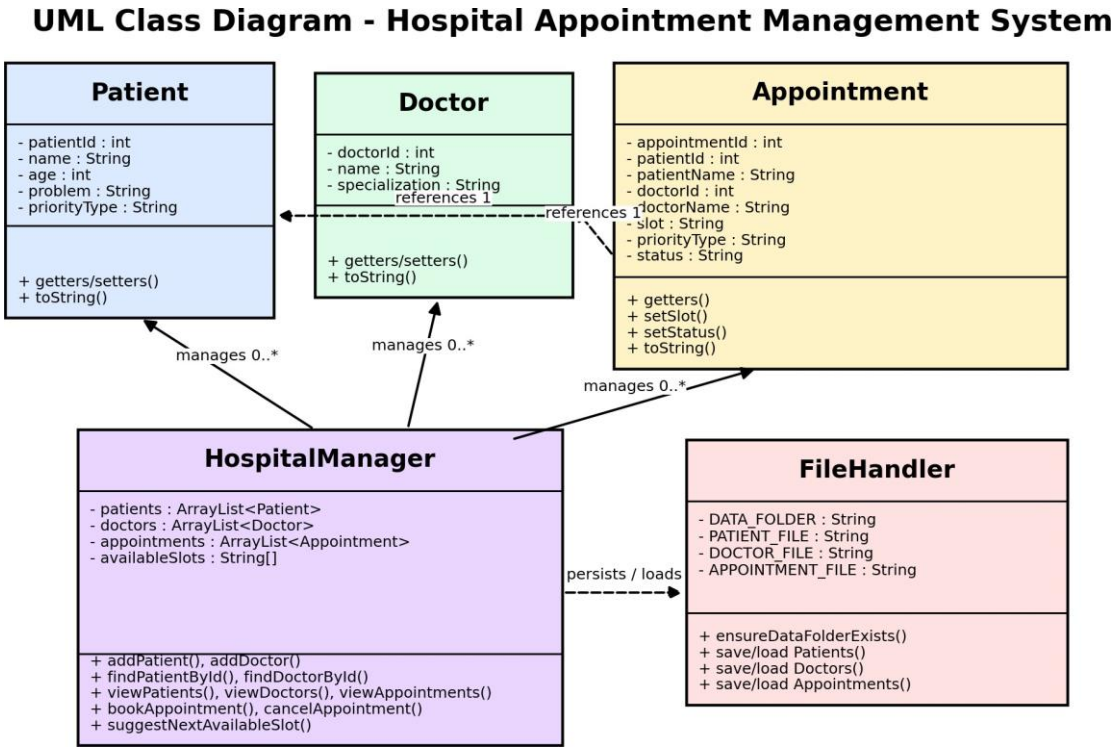

Figure 1. UML class diagram of the Hospital Appointment System.

2.2 Description of Major Classes

Class	Main Responsibility	Key Data / Methods
Patient	Represents a patient registered in the system.	Stores patientId, name, age, problem, priorityType and exposes getter/setter methods.
Doctor	Represents a doctor available for consultation.	Stores doctorId, name, specialization and supports controlled access through methods.
Appointment	Represents one booked, cancelled, or rescheduled appointment.	Stores appointmentId, patient and doctor references by id/name, slot, priorityType, and status.
HospitalManager	Acts as the main service layer that controls the application logic.	Adds records, searches by id, validates slots, handles booking conflicts, schedules/reschedules appointments.
FileHandler	Provides data persistence using text files.	Saves and loads patient, doctor, and appointment data so that records survive after program exit.

2.3 Package Structure

Package / File	Purpose
src/model	Contains the entity classes Patient, Doctor, and Appointment. These classes model real-world objects and keep data representation separate from business logic.
src/service	Contains HospitalManager, the service class that controls validation, booking, viewing, conflict checking, rescheduling, and cancellation.
src/util	Contains FileHandler, which performs save and load operations using basic file persistence.
src/Main.java	Contains the menu-driven entry point. It accepts user input and delegates operations to HospitalManager and FileHandler.

2.4 Interaction Flow

The interaction begins in the **Main** class, where the user selects an operation from the console menu. Depending on the option chosen, the program collects input such as patient id, doctor id, appointment id, or slot.

The request is then forwarded to **HospitalManager**. This class validates whether the patient and doctor exist, checks whether the slot is valid, detects whether another appointment is already booked at the requested time, and applies the priority rules when necessary.

If the user asks to save data or exit the program, **FileHandler** serializes the current lists of patients, doctors, and appointments into text files. At the next start-up, the same class loads those records back into the system.

This layered flow improves clarity because user interaction, business rules, and storage logic are separated instead of mixed into one large file.

3. OOP Concepts Demonstrated

3.1 Encapsulation

Where it is used:

Encapsulation is used in the model classes Patient, Doctor, and Appointment, where fields are declared private and exposed through public methods.

Why it is used:

It protects internal state from uncontrolled direct modification and keeps class data consistent. This is especially useful when the system grows and validation needs to be added later.

```
public class Patient {
    private int patientId;
    private String name;
    private int age;
    private String problem;
    private String priorityType;

    public int getPatientId() {
        return patientId;
    }

    public String getPriorityType() {
        return priorityType;
    }
}
```

Explanation of snippet:

The fields cannot be accessed directly from outside the class. Instead, controlled access is provided through getter methods. This design keeps the patient object secure and clearly structured.

3.2 Abstraction

Where it is used:

Abstraction is visible in HospitalManager, which hides complex booking and conflict-resolution logic behind simple method calls such as bookAppointment() and cancelAppointment().

Why it is used:

Main.java only needs to call service methods; it does not need to know how the scheduling decision is made internally.

```
public String bookAppointment(int appointmentId, int patientId,
                              int doctorId, String slot) {
    Patient patient = findPatientById(patientId);
    Doctor doctor = findDoctorById(doctorId);

    if (patient == null) return "Patient not found.";
    if (doctor == null) return "Doctor not found.";
    if (!isValidSlot(slot)) return "Invalid slot.";

    Appointment existingAppointment =
        findBookedAppointmentByDoctorAndSlot(doctorId, slot);

    // Further conflict-handling logic is hidden here
}
```

Explanation of snippet:

The caller only sees a clean method interface. Internally, the manager performs searching, validation, conflict detection, and priority handling. This is a practical example of abstraction because implementation details are hidden behind a meaningful service method.

3.3 Modularity

Where it is used:

The project is divided into model, service, and util packages, with Main kept as a separate entry point.

Why it is used:

Modularity improves maintainability, readability, and debugging. Each package has a specific responsibility, so future changes can be made with less confusion.

```
package model;   // entity classes

package service; // booking and scheduling logic

package util;    // file save/load support
```

Explanation of snippet:

The package structure itself reflects modular thinking. Instead of mixing all logic into one file, related code is grouped according to purpose.

3.4 Association and Object Collaboration

Where it is used:

HospitalManager collaborates with Patient, Doctor, and Appointment objects, while FileHandler collaborates with lists managed by HospitalManager.

Why it is used:

This mirrors the real problem domain: doctors, patients, and appointments are separate objects, but they interact to fulfill one workflow.

```
private ArrayList<Patient> patients;
private ArrayList<Doctor> doctors;
private ArrayList<Appointment> appointments;

public boolean addDoctor(Doctor doctor) {
    doctors.add(doctor);
    return true;
}
```

Explanation of snippet:

The service class maintains object collections and coordinates collaboration between them. This demonstrates object composition and association in a simple, understandable way.

4. Implementation Highlights

This section focuses on the most important implementation choices that make the project different from a basic CRUD-only hospital system.

4.1 Priority-Based Scheduling Logic

```
int newPriority = getPriorityValue(patient.getPriorityType());
int oldPriority = getPriorityValue(existingAppointment.getPriorityType());

if (newPriority < oldPriority) {
    String nextSlot = suggestNextAvailableSlot(doctorId, slot);

    if (nextSlot != null) {
        existingAppointment.setSlot(nextSlot);
        existingAppointment.setStatus("Rescheduled");
        appointments.add(newAppointment);
        return "Higher priority patient booked. Existing appointment moved.";
    }
}
```

Justification:

This is the core differentiating feature of the project. A normal booking system would simply reject the request if the slot were occupied. Here, the system compares priorities and allows a higher-priority case to take precedence. That makes the design more realistic for hospitals where urgency matters.

4.2 Smart Next-Slot Suggestion

```
public String suggestNextAvailableSlot(int doctorId, String requestedSlot) {
    int startIndex = -1;

    for (int i = 0; i < availableSlots.length; i++) {
        if (availableSlots[i].equalsIgnoreCase(requestedSlot)) {
            startIndex = i;
            break;
        }
    }

    for (int i = startIndex + 1; i < availableSlots.length; i++) {
        if (findBookedAppointmentByDoctorAndSlot(doctorId, availableSlots[i]) == null)
            return availableSlots[i];
    }
    return null;
}
```

Justification:

This snippet improves user experience. Instead of forcing the user to retry manually after a conflict, the system computes the next available slot automatically. This feature also helps demonstrate algorithmic thinking within a beginner-friendly project.

4.3 File Persistence

```
public static void savePatients(ArrayList<Patient> patients) {
    ensureDataFolderExists();
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(PATIENT_FILE))) {
        for (Patient patient : patients) {
            writer.write(patient.getPatientId() + "|" +
                         patient.getName() + "|" +
                         patient.getAge() + "|" +
                         patient.getProblem() + "|" +
                         patient.getPriorityType());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving patients: " + e.getMessage());
    }
}
```

Justification:

Using file persistence makes the system more complete than a temporary in-memory application. Records remain available after the program closes, which increases practical usefulness and demonstrates handling of external resources.

4.4 Menu-Driven User Interaction

```
System.out.println("1. Add Patient");
System.out.println("2. Add Doctor");
System.out.println("3. View Patients");
System.out.println("4. View Doctors");
System.out.println("5. Book Appointment");
System.out.println("6. Cancel Appointment");
System.out.println("7. View Appointments");
System.out.println("8. Save Data");
System.out.println("9. Exit");
```

Justification:

The menu design keeps the project accessible for a beginner while still providing a complete workflow. It also makes testing easier because each feature can be executed independently from the console.

4.5 Controlled Cancellation Instead of Deletion

```
public boolean cancelAppointment(int appointmentId) {
    for (Appointment appointment : appointments) {
        if (appointment.getAppointmentId() == appointmentId &&
            appointment.getStatus().equalsIgnoreCase("Booked")) {
            appointment.setStatus("Cancelled");
            return true;
        }
    }
    return false;
}
```

Justification:

The appointment is not removed from the list; its status is changed to Cancelled. This preserves history and gives the system behavior closer to a real appointment-management process.

5. Testing and Error Handling

Testing was performed manually using the console interface by entering different combinations of valid and invalid input. Particular attention was given to conflict situations because the special feature of the project depends on correct scheduling behavior.

Test Case	Input / Condition	Expected Result	Observed Handling
Duplicate patient id	Add a second patient using an existing patientId.	System should reject the new record.	addPatient() returns false and the user sees that the id already exists.
Duplicate doctor id	Add a second doctor using an existing doctorId.	System should reject the new record.	Doctor registration is blocked in the service layer.
Invalid slot	Book an appointment using a slot outside the predefined slot list.	System should reject the booking.	isValidSlot() returns false and an "Invalid slot" message is shown.

Test Case	Input / Condition	Expected Result	Observed Handling
Normal conflict	Try to book a slot already occupied by an equal- or lower-priority routine booking.	System should suggest the next free slot or reject if none exists.	The application calls suggestNextAvailableSlot() and books the patient into the next open slot if possible.
Emergency override	Emergency patient requests a slot already booked by a lower-priority patient.	System should move the lower-priority booking and assign the requested slot to the emergency case.	Existing appointment status becomes Rescheduled and the emergency patient receives the original slot.
Cancel missing appointment	Cancel an appointmentId that does not exist or is already cancelled.	System should report failure without crashing.	cancelAppointment() returns false and the menu prints an appropriate message.

5.1 Edge Cases Handled

- A booking is attempted before the relevant patient or doctor has been registered.
- The requested slot exists in the menu text but has already been occupied for the selected doctor.
- No next available slot exists after the requested time, so automatic reassignment cannot be completed.
- The same appointment id is used again, which would otherwise create ambiguity in later cancellation or status tracking.

5.2 Failure Scenarios

The most important failure scenario is invalid business input rather than system-level failure. The service class therefore returns meaningful messages such as "Patient not found", "Doctor not found", "Invalid slot", or "Appointment ID already exists".

For file persistence, the application uses try-catch blocks. If file read or write operations fail, the exception message is printed instead of causing an unhandled crash. This is a basic but effective form of defensive programming suitable for the project level.

6. Git Workflow

The project was developed incrementally through multiple commits rather than through a single final upload. This workflow was important because the evaluation criteria explicitly required clear commit discipline and evidence of progressive development.

The early commits focused on the model layer by creating Patient, Doctor, and Appointment. Subsequent commits introduced HospitalManager, the menu-driven Main class, and then the special feature in stages: basic booking, priority conflict checking, next-slot suggestion, rescheduling support, and file persistence.

This sequence shows a sensible engineering progression. First, the data structures were established. Second, the business logic was added. Third, advanced behavior and persistence were integrated. Such a progression makes the repository history understandable both to the instructor and to the developer revisiting the code later.

The commit history also supports debugging. If an issue were introduced during advanced scheduling or file handling, the repository could be inspected step by step to identify the exact stage where the behavior

changed.

Development Phase	Representative Progress
Phase 1 - Data model	Creation of Patient, Doctor, and Appointment classes.
Phase 2 - Core service	Introduction of HospitalManager with add, search, and view operations.
Phase 3 - User workflow	Main class connected to menu-based console interaction.
Phase 4 - Special feature	Priority scheduling, conflict detection, next-slot suggestion, and rescheduling added in separate commits.
Phase 5 - Persistence	FileHandler introduced and load/save support integrated into program startup and exit.

7. Conclusion and Future Scope

The Hospital Appointment System achieves the main objective of the project: it provides a working, menu-driven Java application with a clean modular structure and a feature set that goes beyond a basic beginner-level CRUD program. The inclusion of priority-based scheduling and smart next-slot suggestion gives the project a practical identity and demonstrates that the design was shaped by a real scheduling problem rather than by data entry alone.

From the perspective of object-oriented programming, the project clearly demonstrates encapsulation, abstraction, modularity, and object collaboration. From the perspective of software engineering, it demonstrates incremental development, simple persistence, validation logic, and defensive handling of error cases.

There is still meaningful future scope. A graphical user interface could make the system easier to use. A database such as MySQL or SQLite could replace text-file storage. Doctor-wise calendars, patient history, login roles, date-based scheduling, and notification features could also be added. Even without those additions, the present version is already a coherent and complete project suitable for academic evaluation.

Final Summary: The project combines clear architecture, correct OOP usage, modular packaging, incremental Git development, practical error handling, and a distinct scheduling feature. These qualities make it both understandable for a beginner and strong enough to stand out during evaluation.